

Encoding Hidden Information in Image Files using Steganography

By: Amit Joshi

TJHSST Mobile/Web Research Lab 2017-2018

Abstract: Encoding hidden information in images is widely used in confidential communication and secret data storing. Sensitive information stands a much better chance of going undetected if it is hidden in such a way that from the outside, it doesn't appear to be sensitive information at all.

Goal: An easily accessible secure steganography application

Introduction: Steganography is the practice of encoding hidden information inside a vessel, in this case, an image. By cleverly modifying pixel values slightly, detection can be avoided. My objective is to make a web app for keyed steganography which uses keys, random noise, scrambling, and text encryption to securely hide information in image files.

Background/Theory: While cryptography encodes a message in a way that it is impossible to decode the message without a key or cipher, steganography aims to stop hidden information from being detected altogether. Due to the ability of steganography to avoid suspicion, steganography can even be used in images that are in plain sight. It has been suspected that steganography has been used by terrorist organizations to send messages to one another (Rice University). It makes sense then, that from a defensive standpoint, it is important to be able to recognize and then decode hidden messages by malicious organizations.

At its core, steganography provides another level of security over encryption. Not only can it hide the existence of sensitive information, but it can also enhance the security of already encrypted data (Kawaguchi). This means that data that is already encrypted using cryptography can then be the hidden information that is encoded into an image using steganography.

To implement Steganography on a basic level, I used the Least Significant Bit (LSB) algorithm. First, the secret message needs to be converted to binary. If the value of a color channel of a pixel is written in binary, it will have eight digits. The rightmost one is least significant, and will change the image the least. When read in the right order, these bits form a message. What determines this order is the random seed is the password that the user enters.

References:

Kawaguchi, Eiji. "Applications of Steganography." *Applications of Steganography*. Eigi Kawaguchi, 14 Apr. 2017. Web.

Rice University. "What Is Steganography." *What Is Steganography*. Rice University, n.d. Web.

Development/Techniques: It was very quick to get a basic application of steganography working, where the pixels are modified in a line from left to right. Next, I decided to implement further security measures. First, I implemented keyed steganography. The way this works is that while a user who wishes to encode a message also inputs a password. I then modify this password to create a random seed which I will then use to determine the order in which the pixels need to be modified to encode the secret message. Without the right key, the message cannot be decoded. Next, I focused on obfuscating the message itself. I accomplished this by taking the absolute value of the derivative of the sequence a random number of times. For example, the sequence "011" would become "010" when applying this function one time. I used the random seed that I had created to determine how many times I should apply this function to get the final string to be encoded. For decoding, the message would be integrated the same number of times to get the original series of zeros and ones. My next security measure was the addition of random noise. This means that random pixels get modified in the hopes of confusing outsiders. Approximately 0.1% of the pixels that haven't already been modified are modified by the noise.



Figure 1, two images placed side by side. Left: the vessel image with no encoding. Right: the image with a hidden message of "Prom was so fun!" with the password as "amit"

Technologies: I used Python 2 to write a script that would modify the pixel values to encode a message and simply read pixel values to decode the message. Using the Python Image Library (PIL), I was able to load an image's pixel values, modify them, and save the new image. I used a Node.js server to run the python scripts. Using npm packages such as Child Process I was able to spawn these python processes synchronously. Another npm package I used was file upload, which I used to get the image from the client downloaded onto the server. I also used Director as my IDE.



Conclusion/Future Research: I found that most of the security techniques such as the LSB algorithm, the scattering of the message, and the encryption of the text beforehand were nearly instant. However, the addition of the random noise took roughly four seconds for a 500x500 vessel image. If the frequency of random noise were to be lowered, the time needed for random noise addition would be lowered proportionally. One future idea would be to create a time-sensitive encoded image, which would mean that it can only be decoded between two certain times.

Acknowledgements: I would like to thank my lab director, Mr. Kosek, for all the help that he gave me on this project.

Figure 2, the user interface of my web application